虚拟 DOM 是什么？

- **本质**：用 JS 对象来描述真实 DOM 结构的「快照」。
  - 一个标签对应一个普通的 JS 对象（VNode），记录 `tag` 、 `props` 、 `children` 等。
  - 当某个组件更新时，Vue 会为该组件重新生成一棵虚拟 DOM 子树，然后拿「新子树」和「旧子树」做 diff，最后只对真正变化的部分操作真实 DOM。
- **为什么要用它**
  - 直接操作 DOM 成本高、写起来容易乱；虚拟 DOM 把「视图 = 状态的函数」这个过程交给框架。
  - 通过统一的 diff 和 patch 逻辑，集中优化 DOM 操作。

Vue 中的更新流程（简化）

1. 数据变了（ `reactive` / `ref` / `data` ）。
2. 触发对应组件的 `render` 函数，再次返回一棵新的 VNode 树。
3. `patch(oldVNode, newVNode, container)` ：
   - 比较新旧 VNode
   - 找出哪些需要新增、删除、移动、更新属性/文本
   - 最后对真实 DOM 做最少的变更。

---

diff 算法核心思想

- **同级比较，不跨层级**
  - 只比较同一父节点下的子节点列表。
  - 如果 `tag` 不同（例如 `div` → `span` ），通常直接删旧建新。
- **`key` 的作用**
  - 用 `key` 唯一标识每个子节点（如列表 `v-for="item in list" :key="item.id"` ）。
  - 有了 `key` ，算法能知道「这是同一个节点，只是位置变了 / 内容变了」。
  - 没有 `key` ：按顺序一一对比，插入/删除会导致后面很多节点被误判成「需要重新创建」。
- **子节点 diff（同级 children）大致策略**
  - 从头、从尾同时对比，能快速跳过相同前缀和后缀。
  - 中间「乱序」部分结合 `key` 建立映射，判断：
    - 新列表里有但旧列表没有：创建并插入。
    - 旧列表有但新列表没有：删除。
    - 两边都有：复用、只移动位置或更新内容。
  - Vue 3 会在中间乱序部分使用「最长递增子序列（LIS）」优化，减少 DOM 移动次数。

Vue 2 和 Vue 3 的优化点差异（简要）

- **Vue 2**

- 基于虚拟 DOM 的传统 diff。
- 依赖组件级别的响应式 + 模板编译出的 `render` 函数。
- 每次数据变更对应组件整体做一次虚拟 DOM 树 diff。

- **Vue 3 关键优化**
  - **静态提升（Hoist Static）**：模板中不依赖响应式数据的部分只创建一次，后面复用，不参与 diff。
  - **Block Tree + Patch Flag**：
    - 编译阶段分析出哪些节点会变，给这些节点打「变更标记」（如只会变 text、只会变 props）。
    - 运行时 diff 时只看这些「动态节点」，大量跳过纯静态内容。
  - **更高效的 children diff 算法**（头尾指针 + LIS）。